VNUHCM - UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

★ ☆ ★

CS486 COURSE REPORT

Campus Space Management System

Members of group 05:	MSSV:
Phạm Hữu Nam	24125015
Cao Quảng Hưng	24125010
Nguyễn Hữu Phước	24125018
Trần Đình Quốc Thắng	24125079

TP. Hồ Chí Minh, 06/2026

Mục lục

1 System Overview & LLM Selection

1.1 Problem Statement

The School of Computer Science manages a wide range of shared physical spaces, including classrooms, computer laboratories, project laboratories, meeting rooms, auditoriums, and student workspaces. These spaces are used for teaching activities, seminars, examinations, workshops, research, student projects, and academic events. Because many users need access to the same spaces, the School must ensure that room usage is organized, fair, and efficient.

At present, the booking process is handled manually. Lecturers, teaching assistants, students, and staff usually contact the school office or facility staff by email, phone, or in person. Facility staff then check spreadsheets or shared calendars to verify room availability, user eligibility, required facilities, and maintenance status. This manual process becomes increasingly difficult to manage as the number of booking requests grows. It is time-consuming, error-prone, and may lead to conflicting bookings, incomplete records, and inefficient space utilization.

The School therefore needs a database system to support the management of shared campus spaces in a more structured and reliable way. The system should store information about users, bookable spaces, available facilities, booking requests, approval decisions, check-in and completion records, maintenance activities, and historical usage. It must also enforce important business rules, such as preventing overlapping approved bookings for the same space, blocking bookings for spaces that are under maintenance or closed, and recording accountability information for approvals and maintenance actions.

The main goal of the proposed system is to help the School manage shared campus spaces fairly and efficiently, reduce booking conflicts, prevent the use of unavailable spaces, and preserve complete usage history for future reference and decision-making.

1.2 LLM Models Utilized

- **Big Pickle:** Evaluated via the OpenCode Zen platform, this model is specifically optimized for automated coding agents. Featuring an extensive context window of approximately 200k tokens, it serves as an ideal tool for rapid initial prototyping. However, its primary limitation lies in the absence of advanced reasoning features.
- **DeepSeek V4 Flash Free:** Accessed through the OpenCode platform, this model was deployed to facilitate technical drafting, content refinement, and structural editing (e.g., database design documentation). It demonstrates high efficiency in generating structured explanations and concise academic prose, though it necessitates human oversight to ensure rigorous factual accuracy and alignment with project specifications.

2 Agent Architecture & Directory Structure

2.1 Repository Organization

The repository is separated into requirement sources, reusable project knowledge, agent memory, generated deliverables, execution logs, and OpenCode configuration. Each directory has a specific role, which prevents the agent from mixing source material with final outputs or temporary evaluation artifacts. Table 1 maps the major paths to their responsibilities.

Path	Responsibility
`README.md`	Provides setup instructions, run guidance, and the list of required outputs.
`AGENTS.md`	Defines agent rules and project-specific operating instructions.
`CS486_Project.pdf`	Stores the original assignment brief used as the external project reference.
`req/`	Contains the source business requirements, especially `business-requirement.md`.
`docs/`	Acts as the project knowledge base and workflow constraints, including the overview, tech stack, entity registry, schema registry, and design decisions.
`docs/templates/`	Provides templates and reading or writing guidance for registry-style documentation.
`memory/`	Stores persistent state such as `MEMORY.md`, `ActiveContext.md`, and `Progress.md`.
`outputs/`	Contains final deliverables for Tasks 01–07, from requirement analysis to business query design.
`logs/trajectory/`	Stores task execution traces, grouped by task number.
`logs/eval/`	Stores output evaluation records and future pipeline-level evaluation logs.
`.opencode/commands/`	Defines slash commands for running the full database design pipeline, generating individual tasks, and evaluating results.
`.opencode/skills/`	Stores the main database design pipeline skill, task-specific skills, and the evaluation framework.

Bảng 1: Major repository paths and their responsibilities

2.2 Layered Documentation Strategy

The project adopts an AI-first documentation strategy based on the principle of progressive disclosure [2]. Rather than loading the entire repository into the model context at the beginning of each session, information is organized into multiple layers that are accessed only when relevant to the current task. This design reduces context overload, improves retrieval efficiency, and ensures that stable project knowledge is separated from task-specific procedures and execution history. Table 2 summarizes the four documentation layers used by the agent.

Layer	Primary files	When the agent reads them	Purpose
1	`AGENTS.md`	At the start of the session	Defines global rules, editing constraints, and workflow expectations.
2	`docs/README.md`, `docs/*.md`, `docs/templates/`	On demand, following the documented reading order	Stores domain knowledge, naming conventions, design decisions, entity definitions, schema definitions, and reusable templates.
3	`.opencode/commands/`, `.opencode/skills/`	When a command or task workflow is invoked	Supplies command entry points, pipeline procedures, task rules, and evaluation guidance.
4	`memory/*.md`	During and after task execution	Preserves active context, progress, corrections.

Bảng 2: Progressive documentation layers used by the agent workflow

2.3 Agent Workflow

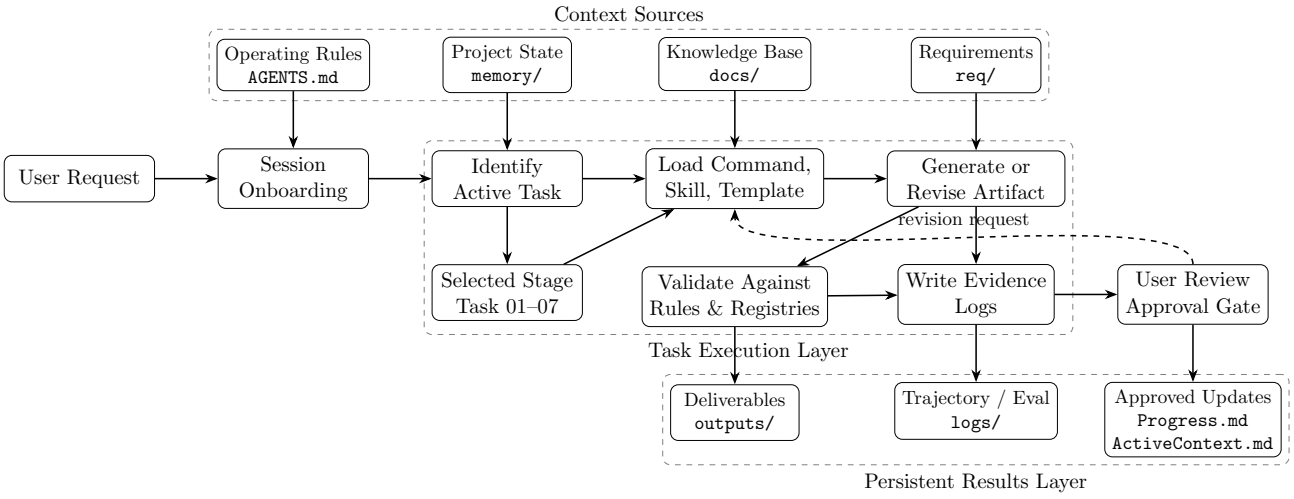
The agent architecture is implemented as a controlled database-design pipeline rather than a single free-form prompting process. When a user invokes a task, the request is first routed to

an OpenCode command, which acts as the workflow entry point. The command identifies the requested stage of the database design lifecycle and loads the corresponding task-specific skill. Examples include business requirement analysis, conceptual database design, logical schema design, database validation, SQL implementation, sample data generation, query design, and evaluation.

Unlike a traditional prompt-based workflow, the system does not rely solely on conversation history to preserve context. Instead, important design decisions are externalized into persistent knowledge artifacts. During execution, the selected skill consults the project knowledge base, reusable templates, and previously generated registries before producing an output. As a result, information discovered in earlier tasks can be reused consistently throughout the pipeline without requiring the model to rediscover the same knowledge multiple times.

The workflow follows a clear separation of concerns. OpenCode commands define task entry points, skills define generation procedures and evaluation rules, the `docs/` directory serves as the authoritative knowledge base, the `memory/` directory maintains persistent project state, the `outputs/` directory stores final deliverables, and the `logs/` directory records execution traces and evaluation evidence. This modular organization improves reproducibility, simplifies auditing, and reduces unintended coupling between different stages of the workflow.

The architecture also incorporates a feedback mechanism. After each task is completed, trajectory logs and evaluation reports are generated and reviewed. Important corrections, unresolved issues, and validated design decisions can then be propagated back into memory and project documentation. Consequently, future tasks benefit from accumulated project knowledge while remaining independent of the original conversation context. Figure 1 illustrates the overall workflow.



Hình 1: Agent Workflow

2.4 Knowledge Propagation Across Tasks

A key design principle of the pipeline is knowledge propagation. Rather than treating each task as an isolated generation step, the system stores important outputs as reusable artifacts that become inputs for subsequent tasks. This approach improves consistency across the database design lifecycle and reduces the risk of contradictory decisions between stages.Knowledge propagation of these artifacts are detailed in Table 3.

Task	Main input(s)	Main output(s)	Role in the pipeline
01	`req/business-requirement.md`	`outputs/01-business-req-analysis-G05.md` `docs/entity-registry.md`	Extracts the business purpose, actors, business rules, candidate entities, attributes, and relationships from the source requirements.
02	`docs/entity-registry.md` `outputs/01-business-req-analysis-G05.md`	`outputs/02-erd-design-G05.md` `updated entity registry`	Converts conceptual entities into an ERD, confirms relationships, cardinalities, participation constraints, and junction entities.
03	`docs/entity-registry.md` `outputs/02-erd-design-G05.md`	`outputs/03-logical-design-G05.md` `docs/schema-registry.md`	Translates the ERD into a normalized relational schema, finalizes names and data types, and populates the schema registry.
04	`docs/entity-registry.md` `docs/schema-registry.md` `outputs/03-logical-design-G05.md`	`outputs/04-design-validation-G05.md` `schema-freeze decision`	Validates requirement coverage, ERD fidelity, normalization, and schema consistency before freezing the schema.
05	`docs/schema-registry.md` `outputs/04-design-validation-G05.md`	`outputs/05-db-definition-G05.sql` `DDL compile logs`	Generates SQL Server DDL from the frozen schema, including tables, constraints, indexes, triggers, and validation-supporting database logic.
06	`outputs/05-db-definition-G05.sql` `docs/schema-registry.md` `docs/entity-registry.md`	`outputs/06-sample-data-G05.sql` `logs/execution/task06/` `logs/eval/task06/`	Produces realistic, idempotent sample data and expected-error tests that prove key business rules against the approved schema.
07	`outputs/05-db-definition-G05.sql` `outputs/06-sample-data-G05.sql` `outputs/01-business-req-analysis-G05.md` `req/business-requirement.md`	`outputs/07-query-design-G05.sql` `logs/eval/task07/`	To produce personalized, meaningful, parameterized, and validated T-SQL queries based on student ideas that directly resolve core operational and analytic questions from role-based perspectives without loss of prior work.

Bảng 3: Task-specific artifacts used across the database design pipeline

3 Evaluation & Agent Improvement Process

3.1 Rubric-Based Evaluation Criteria

The evaluation framework was designed to measure both the quality of the generated database artifacts and the quality of the agent process used to produce them. This separation follows agent-evaluation guidance that recommends assessing not only final answers, but also reasoning traces, tool use, and intermediate actions [3, 4, 5, 6]. Therefore, each task was evaluated with two independent scores: an **artifact quality score** and an **agent process quality score**. The two scores were intentionally not merged, because a task can produce a correct deliverable while still using an incomplete, inefficient, or unsafe workflow.

Artifact quality rubric. Artifact quality was scored from 0 to 5 using a weighted database-design rubric. A score of 0 means the criterion is missing or seriously wrong, while a score of 5 means the output is strong, complete, consistent, and well justified. The weighted artifact score is calculated as the sum of each criterion score multiplied by its weight.

Criterion	Weight	Evaluation focus
Completeness of required outputs	10%	Required files exist and contain all mandatory sections for the active task.
Requirement coverage	20%	Source requirements and business rules are preserved, especially booking conflicts, unavailable-space restrictions, status constraints, and maintenance rules.
Database design correctness	20%	Entities, relationships, cardinalities, tables, keys, constraints, triggers, and indexes are technically reasonable and enforce the intended rules.
Traceability	15%	Requirements can be followed through business rules, entities, relationships, tables, constraints, sample data, and queries.
Assumptions and open questions	10%	Missing or ambiguous information is explicitly recorded instead of being silently invented.
Cross-file consistency	10%	Entity names, table names, attributes, statuses, and constraints remain consistent across outputs, registries, and design decisions.
SQL Server compatibility	10%	SQL artifacts use valid Microsoft SQL Server syntax and provide execution evidence where required.
Query usefulness	5%	Final queries answer realistic operational needs such as booking history, utilization, no-shows, maintenance, and unavailable spaces.

Bång 4: Artifact quality rubric used for generated database design outputs

Agent process rubric. Process quality was evaluated from the trajectory logs rather than from the final artifact alone. This matches trajectory-based evaluation for multi-step agents, where silent process failures can occur even when the final answer appears plausible [4, 6]. After generating or revising an `outputs/0X-*` artifact, the agent records the plan, actual execution steps, files read and written, deviations, outcome, and self-detected fixes. The six process metrics are scored independently from 0 to 5 and then averaged into a process score.

Metric	Evaluation focus
PlanQuality	Whether the plan is logical, ordered, complete, and suited to the active pipeline task.
PlanAdherence	Whether actual execution followed the plan or justified deviations clearly.
StepEfficiency	Whether the agent avoided redundant reads, writes, retries, and unnecessary context loading.
TaskCompletion	Whether the required output exists and satisfies the task's success criteria.
ToolCorrectness	Whether the agent read and wrote the expected files while avoiding forbidden paths.
ArgumentCorrectness	Whether commands and file targets used correct paths, group suffixes, and read/write directions.

Bång 5: Agent process metrics scored from trajectory logs

Evaluation workflow. The recommended workflow is iterative: the agent first generates a task artifact, then writes a trajectory log, then the evaluator runs a task-level evaluation when a quality check is needed. The evaluation report returns an artifact score, a process score, supporting evidence, and the top improvement actions. Those results are then used to revise the output, template, command, or task-specific skill when needed. At larger milestones, a pipeline-level evaluation checks cross-task consistency. This reflects the practice of combining end-to-end artifact evaluation with component-level checks for tool choice, file access, arguments, and side effects [5, 6, 7].

3.2 Agent Improvement Techniques

3.2.1 Task-Scoped Progressive Context Disclosure

The first improvement technique was **task-scoped progressive context disclosure**. Instead of loading the entire repository, the agent followed the documented reading order and opened only the files relevant to the active pipeline stage. The process started with project rules and memory files, then moved to task-specific skills, templates, and the exact upstream artifacts required for the current deliverable.

This technique matched the seven-task pipeline. For Task 01, the authoritative input was `req/business-requirement.md`, so the agent avoided premature schema assumptions. For Tasks 02 and 03, the context expanded to the entity registry, ERD, business-rule analysis, and technical conventions. After schema freeze, Tasks 05 and 06 relied primarily on the locked schema registry, approved validation output, DDL, and task-specific SQL guidance.

This approach aligns with the progressive disclosure principle [9] and improved the agent in three ways: **First**, it reduced context pollution by excluding unrelated files. **Second**, it lowered cross-task drift via controlled inputs. **Third**, it simplified evaluation through verifiable logs.

3.2.2 Command and Skill Metadata with YAML Front Matter

The second improvement technique was the use of **YAML front matter** as a small machine-readable header before longer human-readable instructions. YAML front matter is a common pattern for storing file-level metadata at the beginning of Markdown-like documents [10]. In this project, it appeared in command files, skill files, and trajectory logs.

In `.opencode/commands/`, front matter helped identify command-level properties before the prompt body was read. In `.opencode/skills/`, it exposed `name` and `description` fields

that helped the agent select task-specific behavior. In `logs/trajectory/`, it standardized run metadata such as `task`, `group`, `run_at`, `status`, and `revision_of`. OpenCode's command and skill documentation directly supports this style of command routing and skill discovery [11, 12].

This improved the agent in three ways: **First**, it strengthened command routing through early command distinction. **Second**, it enhanced skill discovery without loading full task files. **Third**, it increased evaluation reliability via stable trajectory identifiers. To prevent metadata drift, schema facts and business rules were strictly maintained in authoritative registries rather than front matter.

3.2.3 Registry-Grounded Source-of-Truth Control

The third improvement technique was **registry-grounded source-of-truth control**. Stable database knowledge was externalized into persistent documentation: `docs/entity-registry.md` for conceptual entities and attributes, `docs/schema-registry.md` for relational tables and constraints, and `docs/design-decisions.md` for rationale and trade-offs.

This structure matched the project pipeline. Task 01 populated the entity registry from the business requirement. Task 02 refined relationships and cardinalities. Task 03 locked attributes and populated the schema registry. Task 04 validated both registries before schema freeze. After that point, Tasks 05–07 had to treat the registries as read-only control documents rather than silently modifying upstream design facts.

This improved the agent in three ways. **First**, it reduced context drift because later tasks did not need to rediscover the entity model from scratch. **Second**, it improved cross-file consistency because ERD, logical schema, DDL, sample data, and future query design could be checked against the same approved registries. **Third**, it introduced change control: if a later task needed to revise a design choice, the reason had to be recorded in `docs/design-decisions.md` rather than hidden inside a generated output.

This approach aligns with both **codified context infrastructure** and **source-of-truth synchronization** in agent systems. In our project, the registries acted as **domain-specific cold memory**, while the design-decision log served as a **governance record**. Instead of allowing duplicated local representations to drift, the agent must consult these canonical registries before producing or validating downstream artifacts. Together, this framework made the workflow more **auditable, reproducible, and resistant to hallucinated schema changes**.

3.2.4 Requirement-to-Artifact Traceability Control

The fourth improvement technique was **requirement-to-artifact traceability control**. Each important requirement needed a visible path through the pipeline: from the original business requirement, to numbered business rules, entities, relationships, logical tables, database constraints, DDL triggers, sample-data tests, and eventually business queries. This made it easier to verify that requirements were preserved instead of being dropped or reinterpreted across tasks.

In this project, traceability appeared as a distributed control mechanism rather than a single perfect matrix. Task 01 established numbered business rules and assumptions. The entity registry stored source references for conceptual entities and attributes. The schema registry mapped business rules to database-level enforcement mechanisms such as constraints, indexes, and triggers. Task 06 then strengthened downstream traceability by labeling expected-error cases with BR numbers and preserving SQL execution evidence.

This improved the agent in three ways. First, it enabled coverage checking by exposing missing requirement links. Second, it facilitated backward debugging by tracing design errors back to source rules. Third, it enhanced auditability by making the requirement's entire route

inspectable. This is consistent with requirements traceability research, which treats trace links as a way to follow requirements forward and backward through development artifacts [13].

4 Task-by-Task Evaluation Log

4.1 Task 01: Business Requirement Analysis

4.1.1 Input Used

Task 01 analysed the project requirements to establish a complete understanding of the business domain before any design activities. The primary source of information was `req/business-requirement.md`, supported by project documentation defining the business scope, technology context, entities, design decisions, and current working state.

The following inputs were used:

- `req/business-requirement.md` — primary source of requirements
- `docs/project-overview.md` — project scope
- `docs/tech-stack.md` — implementation context
- `docs/entity-registry.md` — candidate entities
- `docs/design-decisions.md` — design conventions
- memory files and project instructions — task context

4.1.2 Output Produced

The task produced `outputs/01-business-req-analysis-G05.md`. The output summarised the project purpose, six user roles, core domain objects, booking and approval workflows, maintenance handling, 14 business rules, assumptions, open questions, and a requirements traceability matrix linking business rules back to `req/business-requirement.md`.

4.1.3 Issues and Improvements

Task 01 required three evaluation rounds before approval. The initial version captured the core business requirements but lacked several documentation and traceability elements required by the evaluation framework. Table 6 summarises the major issues identified, the corresponding improvements introduced, and their effect on the final artifact.

Issue Found	Improvement Made	Effect
The initial version lacked a requirements traceability matrix.	A 14-row traceability matrix was added, mapping each business rule to source evidence and classifying it as explicit, inferred, or design-convention based.	Improved requirements traceability and source coverage.
Incident reporting was not analysed as a separate concern.	The analysis was expanded to explicitly include incident reporting within the business requirements.	Improved completeness of business process coverage.
Entity relationships did not include explicit cardinality notation.	Entity relationships were revised to include explicit cardinality notation.	Improved clarity of relationship analysis for subsequent design tasks.
Related formatting and task state updates were incomplete.	Formatting and memory updates were completed, and Task 01 was marked approved before proceeding to Task 02.	Improved workflow consistency and task progression.

Bång 6: Issues identified and improvements made during Task 01 iterations

4.1.4 Evaluation Result

Task 01 was evaluated across three rounds using both artifact-level and process-level criteria. The final artifact score was **4.40/5.00**, while the process score reached **4.50/5.00**. All eight file-level checks passed, including stakeholder coverage, process coverage, constraints, candidate entities, relationships, assumptions, open questions, and traceability.

The iterative refinement primarily strengthened documentation completeness, requirements traceability, and relationship analysis, resulting in an approved business requirements specification that served as the foundation for subsequent design tasks.

4.2 Task 02: Conceptual Database Design

4.2.1 Input Used

Task 02 transformed the approved business analysis into a conceptual entity-relationship model. The primary source of information was `docs/entity-registry.md`, while `outputs/01-business-req-analysis-G05.md` was consulted only to resolve requirement ambiguities. Memory files and project instructions provided additional task context.

The following inputs were used:

- `docs/entity-registry.md` — primary source of entity definitions
- `outputs/01-business-req-analysis-G05.md` — fallback for ambiguity resolution
- memory files — session context
- project instructions

4.2.2 Output Produced

The task produced `outputs/02-erd-design-G05.md`. This file provides an overview of the core entities and presents a Crow's Foot ER diagram in Mermaid format. After the later booking-workflow refinement, the conceptual model aligns with 9 finalized entities, including separate approval and session records for booking decisions and actual usage. It also enhances the Rela-

tionship Participation Table in `docs/entity-registry.md` by adding two columns for Mermaid notation and participation explanations.

The output further documents logical constraints enforced at the application or trigger level that cannot be represented in Mermaid ERD syntax. Additionally, `docs/design-decisions` was updated.

4.2.3 Issues and Improvements

Task 02 required three iterations before reaching a fully compliant conceptual ERD. The first implementation established the overall structure, while later iterations addressed documentation completeness and conceptual-model purity. Table 7 summarises the major issues identified, the corresponding improvements introduced, and their effect on the final artifact.

Issue Found	**Improvement Made**	**Effect**
The initial artifact lacked four mandatory sections required by the specification: *Relationship Participation Summary*, *Logical Constraints*, *Design Decisions*, and the *Pre-Submission Validation Checklist*.	All four missing documentation sections were added during the second iteration, achieving full functional and structural completeness.	Improved compliance with the required artifact structure.
The ERD contained physical database concepts, including SQL datatypes, audit columns, and foreign key markers.	The final iteration enforced strict conceptual purity by replacing physical types with generic types, removing audit columns from all nine entities, and eliminating all foreign key markers.	Produced a fully conceptual ERD aligned with the design rules.
The initial workflow did not verify the generated output against the required skill output specification before completion.	The underlying workflow and `SKILL.md` rules were revised to enforce the required conceptual-model constraints.	Improved alignment between the generation process and the conceptual design requirements.

Bång 7: Issues identified and improvements made during Task 02 iterations

4.2.4 Evaluation Result

Task 02 was evaluated using both artifact-level and process-level criteria across three iterations. Detailed scores for each evaluation round are summarised in Table 8.

Evaluation Area	Score	Evidence / Comment
Iteration 1 — artifact	5.00 / 5.00	Initial evaluation awarded full artifact marks despite missing required documentation sections.
Iteration 1 — process	5.00 / 5.00	Initial process evaluation awarded full marks.
Iteration 2 — artifact	4.85 / 5.00	Required documentation sections were added, but conceptual purity issues remained in the ERD.
Iteration 2 — process	4.67 / 5.00	Evaluation became stricter, explicitly penalising assumptions documentation and implicit traceability gaps.
Iteration 3 — artifact	4.75 / 5.00	Conceptual purity was fully achieved by removing physical database constructs from the ERD.
Iteration 3 — process	4.50 / 5.00	An additional process deduction was applied for reading downstream artifacts outside the expected Task 02 input scope.

Bång 8: Task 02 evaluation scores across three iterations

The evaluation scores decreased across the three iterations, not because the artifact regressed, but because the evaluation criteria became progressively more rigorous. While the first evaluation overlooked documentation and traceability gaps, later evaluations explicitly assessed those aspects and introduced an additional process deduction for reading downstream artifacts outside the expected Task 02 input scope. At the same time, the artifact itself continued to improve, first by completing all required documentation sections and finally by achieving full conceptual purity.

4.3 Task 03: Logical Database Design

4.3.1 Input Used

Task 03 transformed the approved conceptual database design into a complete logical database schema for Microsoft SQL Server. The primary inputs were the business analysis from Task 01, the conceptual ERD from Task 02, together with the entity and schema registries, technical conventions, design decisions, and the original business requirements. These artifacts provided the entities, relationships, SQL Server rules, naming conventions, and unresolved questions required to produce the logical schema.

The following inputs were used:

- `outputs/01-business-req-analysis-G05.md` — business analysis
- `outputs/02-conceptual-design-G05.md` — conceptual ERD
- `docs/entity-registry.md` — entity definitions
- `docs/schema-registry.md` — schema registry
- Technical conventions, design decisions, and original business requirements

4.3.2 Output Produced

The task produced `outputs/03-logical-design-G05.md`, which defined a complete logical database schema. The final version contained 9 normalized tables after splitting booking decisions and actual usage sessions into `booking_approvals` and `booking_sessions`. It also documented primary and foreign keys, SQL Server data types, enum checks, indexes, trigger-based business rule enforcement, normalization proof, naming conventions, ERD deviations, and resolutions for all five open questions identified during Task 01.

4.3.3 Issues and Improvements

Task 03 required several refinements before reaching the final logical design. The initial version satisfied the overall logical schema requirements, but evaluation identified several issues related to business-rule enforcement, document formatting, and workflow discipline. Table 9 summarises the issues found, the corresponding improvements, and their effect on the final deliverable.

Issue Found	**Improvement Made**	**Effect**
BR8 and BR9 were initially treated as application-level rules instead of database-enforced rules.	The logical design separated actual usage tracking into `booking_sessions` and added trigger-level rules for check-in, completion time, and space condition recording.	Ensured actual time and condition tracking were represented as database-level rules instead of informal application behavior.
One Markdown table contained a column separator mismatch.	The table formatting issue was corrected.	Improved document consistency and formatting correctness.
One revision updated a Task 04 file outside the expected Task 03 write scope.	The logical design revision was updated to v1.2 and the validation report was synchronized accordingly.	Restored consistency between the logical design and validation artifacts while keeping the documented business-rule enforcement aligned.

Bång 9: Issues identified and improvements made during Task 03 iterations

4.3.4 Evaluation Result

Task 03 achieved a perfect applicable artifact score of **4.75/4.75**, demonstrating complete logical schema coverage, requirement traceability, database correctness, consistency, SQL Server compatibility, and completeness. The process score was **4.67/5.00**. All file-level checks passed, including verification of tables, columns, keys, constraints, indexes, trigger logic, normalization, and naming conventions.

The final logical design fully satisfied the applicable artifact evaluation criteria, while the remaining process deductions reflected workflow issues rather than deficiencies in the logical database schema itself.

4.4 Task 04: Database Design Validation

4.4.1 Input Used

Task 04 validated the logical database design before schema freeze by checking all approved upstream artifacts for structural consistency, business-rule coverage, normalization, and SQL Server compliance. The validation was performed against the approved business analysis, conceptual design, logical design, registries, design decisions, and technical conventions.

The following inputs were used:

- `outputs/01-business-req-analysis-G05.md` — business rule reference (BR1–BR14)
- `outputs/02-erd-design-G05.md` — conceptual ERD
- `outputs/03-logical-design-G05.md` — logical database design
- `docs/entity-registry.md` — entity definitions and attributes
- `docs/schema-registry.md` — normalized schema registry

- `docs/design-decisions.md` — architectural decisions
- `docs/tech-stack.md` — SQL Server conventions and standards

4.4.2 Output Produced

The task produced `outputs/04-design-validation-G05.md`, a comprehensive validation report covering the complete logical database design.

The report validated the expanded business-rule set, audited all 9 relational tables, verified the revised relationship mapping, recorded design inconsistencies, and concluded with **Schema Freeze Approved** after the booking workflow was normalized into separate booking, approval, and session records.

The validation document included ERD fidelity analysis, business-rule coverage, key adequacy assessment, relationship verification, constraint completeness, naming consistency checks, Third Normal Form verification, index strategy review, prioritized issue logs, registry status, self-checklists, idempotency verification, and final schema validation.

4.4.3 Issues and Improvements

Task 04 was refined over three validation iterations to improve consistency, database-level enforcement, and cross-file synchronization. Table 10 summarizes the issues identified, the corresponding improvements, and their effects on the final validation report.

Issue Found	**Improvement Made**	**Effect**
Missing performance indexes in the schema registry and one index naming discrepancy across registries.	Four missing indexes were added to `schema-registry.md`, and the inconsistent index name was unified as `idx_bookings_time_range`.	Improved index completeness and cross-registry consistency.
Three minor constraint gaps remained for facility quantity, task completion time, and actual booking end time.	The gaps were documented and intentionally deferred to the DDL implementation stage in Task 05.	Preserved validation completeness while assigning implementation to the appropriate development stage.
BR8 and BR9 were still treated too much as application-level logic, and the approval/check-in responsibilities were concentrated in `bookings`.	The validation was updated to support the split design: `booking_approvals` records approval decisions and rejection reasons, while `booking_sessions` records check-in, actual time, and condition information. The report also added self-check and idempotency sections.	Strengthened database-level rule enforcement and clarified the separation of booking request, approval, and usage-session responsibilities.
A naming mismatch existed between the logical design and schema registry for the maintenance table.	The schema registry standardized the physical table name as `maintenance`; related indexes, foreign keys, and downstream references were re-verified before schema freeze.	Completed cross-file synchronization with the final 9-table schema and removed the previous singular/plural ambiguity.

Bång 10: Issues identified and improvements made during Task 04 iterations

4.4.4 Evaluation Result

Task 04 achieved a weighted evaluation score of **4.65/5.00**. The validation demonstrated complete requirement coverage by verifying the expanded business-rule set, confirmed the normalized 9-table schema, maintained clear traceability across upstream artifacts, documented assumptions and remaining constraints, verified SQL Server compatibility, and improved query usefulness through additional performance indexes.

The remaining deductions reflected minor completeness and cross-file naming issues rather than structural deficiencies in the validated database design.

4.5 Task 05: Database Implementation

4.5.1 Input Used

Task 05 transformed the approved logical design into a deployable Microsoft SQL Server implementation. The primary source of truth was `docs/schema-registry.md`, which had already passed validation and reached schema-freeze status. Earlier task outputs were consulted only as supporting references to preserve end-to-end traceability from requirements to ERD, logical design, validation, and final DDL.

The following inputs were used:

- `docs/schema-registry.md` — primary source of truth (locked)
- `outputs/04-design-validation-G05.md` — validation evidence
- `outputs/03-logical-design-G05.md` — trigger logic and resolved ambiguities
- `outputs/01-business-req-analysis-G05.md` — business rule origin
- `memory/Progress.md`, `memory/ActiveContext.md` — session state

4.5.2 Output Produced

The task produced `outputs/05-db-definition-G05.sql`, a complete SQL Server DDL script implementing the physical schema across nine tables: `departments`, `users`, `spaces`, `facilities`, `space_facilities`, `bookings`, `booking_approvals`, `booking_sessions`, and `maintenance`.

The script included primary keys, foreign keys, unique constraints, check constraints, default values, supporting indexes, and trigger logic enforcing 19 business rules from the schema registry. Key database-level enforcements included: interval overlap prevention for approved bookings, space availability checks, room capacity validation, maintenance-period booking blocks, approval and rejection metadata requirements, and check-in and completion field enforcement via status-transition triggers.

The script compiled on SQL Server 2019 with zero errors, producing 7 tables, 10 foreign keys, 13 CHECK constraints, 7 primary keys, and 18 DEFAULT constraints as confirmed in `logs/eval/task05/`.

4.5.3 Issues and Improvements

Task 05 required three rounds of refinement. The first implementation was structurally correct and compiled successfully, but evaluation identified weaknesses in process discipline, traceability documentation, and persistent verification evidence. Table 11 summarises the major issues found, the corresponding improvements introduced in later iterations, and their effect on the implementation and workflow.

Issue Found	**Improvement Made**	**Effect**
Expected context files were not recorded in the task trajectory.	The generation workflow was revised to explicitly read and record all required inputs: memory files, schema registry, design decisions, validation report, and logical design artifacts.	Improved process traceability and ToolCorrectness score.
Auto-stamp triggers lacked explicit recursion-protection guards.	An `IF NOT UPDATE(updated_at) RETURN` guard was added to all six updated_at triggers, providing defense-in-depth beyond the SQL Server default `RECURSIVE_TRIGGERS OFF` setting.	Eliminated the file-level check failure for infinite recursion prevention.
Compile evidence was not consistently persisted at the expected path.	Later iterations explicitly saved compile verification logs under `logs/eval/task05/` using the `YYYY-MM-DD-HHmm` naming convention.	Improved auditability and reproducibility across evaluation runs.
Business-rule enforcement was present but not easy to trace from DDL alone.	Trigger comments, validation messages, and inline documentation were aligned with the corresponding business rules (BR) defined in the schema registry. (e.g. `- BR1:`, `- BR6:`).	Strengthened traceability from requirements to physical implementation.
Minor integrity checks recommended by Task 04 were missing from initial DDL.	Additional CHECK constraints were added for `quantity`, `completion_time`, and `actual_end_time` as explicit safeguards, documented with `- Additional integrity safeguard` comments.	Improved defensive database integrity while preserving schema-registry compliance.

Bång 11: Issues identified and improvements made during Task 05 iterations

4.5.4 Evaluation Result

Task 05 was evaluated using both artifact-level and process-level criteria across three iterations. The artifact score measured DDL quality, while the process score measured workflow discipline, file usage, trajectory evidence, and compile-log persistence.

Evaluation Area	Score	Evidence / Comment
Iteration 1 — artifact	4.40 / 5.00	Complete and compilable DDL. Deductions for incomplete assumptions documentation and missing traceability references inside the script.
Iteration 1 — process	4.17 / 5.00	Task completed successfully, but four expected common reads (AGENTS.md, memory/MEMORY.md, docs/project-overview.md, outputs/01-business-req-analysis-G05.md) were absent from the trajectory record, penalising ToolCorrectness.
Iteration 2 — artifact	4.90 / 5.00	Full artifact quality achieved: robust trigger safeguards, complete BR comments, strict schema-registry compliance, and zero compile errors confirmed.
Iteration 2 — process	4.33 / 5.00	Workflow improved significantly; compile-log persistence still incomplete, preventing full process compliance.
Iteration 3 — artifact	4.90 / 5.00	Artifact correctness maintained. Command files simplified; all task-specific behavior moved into the dedicated skill.
Iteration 3 — process	4.50 / 5.00	Pre-generation schema-compliance audit enforced; required context files verified before generation; compile logs and trajectory consistently persisted. Remaining deduction for PlanAdherence: first compile failed due to stale objects from a previous run, recovered cleanly via DROP/CREATE but not flagged as a plan deviation in the trajectory.

Bảng 12: Task 05 evaluation scores across three iterations

The iterative development cycle brought the artifact score from **4.40** to a stable **4.90**, while the process score improved from **4.17** to **4.50**. The remaining process deduction reflected a minor trajectory-recording gap rather than any correctness issue in the generated schema.

4.6 Task 06: Sample Data Preparation

4.6.1 Input Used

Task 06 used the frozen Task 05 DDL as the primary source of truth and converted the approved schema into executable SQL Server sample data.

The following inputs were used:

- `outputs/05-db-definition-G05.sql` — locked SQL Server DDL
- `docs/schema-registry.md`, `docs/entity-registry.md` — read-only schema and entity registries
- `outputs/01-business-req-analysis-G05.md`, `req/business-requirement.md` — business-rule source
- `.opencode/commands/06-generate-sample-data.md` — command interface
- `.opencode/skills/db-design-pipeline/06-sample-data/SKILL.md` and its `references/` files — generation, idempotence, execution, and completion workflow

4.6.2 Output Produced

The task produced `outputs/06-sample-data-G05.sql`, a SQL Server sample data script for the Campus Space Management System. The final version contains an assumptions header, `SET QUOTED_IDENTIFIER ON`, an idempotent cleanup-and-reseed section, FK-safe inserts, intentional expected-error cases, and verification queries.

The final script seeds all nine schema tables: `departments`, `users`, `spaces`, `facilities`, `space_facilities`, `bookings`, `booking_approvals`, `booking_sessions`, and `maintenance`. It covers all required user roles, account statuses, space types, space statuses, booking statuses, booking purposes, and maintenance statuses. The final seed set includes 9 spaces, 8 Task 06 users, maintenance records, booking approvals, booking sessions, and valid booking scenarios, 13 expected-error cases, and 13 verification queries.

4.6.3 Issues and Improvements

Task 06 evolved through several iterations. The most important process lesson was that placing too much procedural detail directly inside the command and main skill made the agent focus on producing the SQL artifact while treating trajectory recording as a late or easily missed step. The fix was to use progressive disclosure: keep the command as a small interface, keep the main skill as the task router, and move phase-specific instructions into focused reference files that are read only when needed.

Issue Found	**Improvement Made**	**Effect**
Workflow guidance and evidence capture were too scattered: the command and skill held too much procedural detail, and one trajectory treated execution validation as a separate manual step.	Task 06 was reorganized into progressive-disclosure reference files: `data-coverage.md`, `idempotence-and-test-isolation.md`, `business-rule-proofs.md`, `sql-style-and-ordering.md`, `execution-validation.md`, and `completion-actions.md`. The completion reference was strengthened to require planned inputs, final `sqlcmd` verification, execution-log review, documented fixes, and a final trajectory.	The workflow became easier to audit. Trajectory writing, execution review, and rerun expectations became explicit completion gates, and the final trajectory captured the debugging history with the successful execution log.
The early data script was not reliably rerunnable, and expected-error cases could fail for setup reasons instead of proving the intended constraints.	A cleanup-and-reseed strategy was added using stable `T06-` space codes, `t06.` emails, reverse-FK-order deletion, stable natural-key lookups, prerequisite checks, and targeted `TRY/CATCH` blocks.	The script became rerunnable against a database containing previous Task 06 data, and expected-error cases began proving the intended rules: BR1 overlap, BR2 unavailable spaces, BR3 capacity, BR4 maintenance overlap, BR6/BR7 metadata, BR8/BR9 lifecycle fields, and key declarative constraints.
Final execution still exposed SQL Server runtime issues: `QUOTED_IDENTIFIER` was effectively off, lookup variables crossed `GO` batch boundaries, BR8/BR9 tests hit BR4 first, and duplicate `space_facilities` rows were inserted.	`SET QUOTED_IDENTIFIER ON` was added, lookup variables were redeclared inside each batch, BR8/BR9 cases were moved to a space without active maintenance, and duplicate facility-link inserts were removed.	Execution became compatible with the Task 05 filtered index. The final log showed all valid sections completing and all 13 expected-error cases catching the intended rule-specific errors.

Bång 13: Issues identified and improvements made during Task 06 iterations

4.6.4 Evaluation Result

Task 06 was evaluated across four recorded rounds. Early iterations produced a useful sample-data artifact but exposed important gaps in idempotence, expected-error isolation, and trajectory completeness. Later iterations improved both the SQL artifact and the agent workflow until the final evaluation reached full score.

Evaluation Area	Score	Evidence / Comment
Iteration 1 — artifact	4.10 / 5.00	The script covered all major tables and scenarios, but the first execution against pre-existing data produced duplicate-key and `NULL` cascade errors. Many expected-error cases therefore caught setup failures rather than the intended business rules.
Iteration 1 — process	4.67 / 5.00	The trajectory listed the main generation steps and correct files, but task completion was penalized because execution validation exposed unresolved failures.
Iteration 2 — artifact	4.90 / 5.00	The script was overhauled with idempotent cleanup, guarded inserts, stable lookups, and corrected expected-error isolation. The revised execution log showed clean valid seed sections and intended rule failures.
Iteration 2 — process	4.67 / 5.00	The artifact worked, but the trajectory did not fully document the idempotence revision or make execution validation part of the original plan.
Iteration 3 — artifact	4.90 / 5.00	`SET QUOTED_IDENTIFIER ON` and rerun evidence strengthened SQL Server compatibility and idempotence. All 13 expected-error tests were captured correctly.
Iteration 3 — process	4.83 / 5.00	The trajectory documented the `QUOTED_IDENTIFIER` failure and fix, but the evaluation still recommended a clearer assumptions section and fuller file touch traceability.
Final iteration — artifact	5.00 / 5.00	The final script included assumptions, FK-safe ordering, cleanup-and-reseed idempotence, all required status/type coverage, 13 expected-error cases, and verification queries for audit, soft-delete, maintenance, no-show, upcoming bookings, and history reporting.
Final iteration — process	5.00 / 5.00	The final trajectory documented planned reads, reference-file usage, two execution cycles, self-detected errors, fixes, and the successful final `sqlcmd` log at `logs/execution/task06/2026-06-22-0948-output.txt`.

Bång 14: Task 06 evaluation scores across four iterations

Overall, Task 06 improved from an artifact score of **4.10** to **5.00**, while the process score improved from **4.67** to **5.00**. The final remaining recommendations were minor auditability improvements: explicitly list common context reads in the trajectory, run one additional idempotence proof execution, and document the intentionally retained pending rows created during some expected-error setup cases.

4.7 Task 07: Query Design

4.7.1 Input Used

Each student read the following inputs before designing their queries:

- `outputs/05-db-definition-G05.sql` — physical schema reference

- `outputs/06-sample-data-G05.sql` — sample data for realistic queries
- `req/business-requirement.md` — business context and user roles
- `outputs/01-business-req-analysis-G05.md` — business rules fallback

4.7.2 Output Produced

The task produced `outputs/07-query-design-G05.sql`, containing all student queries assembled into a single executable file. Each query follows the 4-field template: business question, target user(s), explanation of usefulness, and a parameterized T-SQL statement using `DECLARE` blocks.

4.7.3 Agent Configuration and Improvement Process

Initial approach. The initial design treated Task 07 as a straightforward generation task: the agent would read the approved schema and sample data, then produce five queries independently. A basic command and skill were written to define the output template and SQL quality rules. While the agent successfully generated syntactically valid queries, the output lacked depth and variety. Queries tended to cluster around simple availability checks and repeated similar patterns across students, resulting in low semantic coverage of the domain's business questions.

Improvement: role-scoped query generation. After reviewing the initial output, the team identified the root cause: the agent had no guidance on which user perspective each student should adopt. Without role constraints, every student's queries converged toward the same generic availability-search pattern.

The command was extended with explicit per-student arguments:

- `-student-name` to assign authorship to each query block
- `-target-users` to restrict each student's queries to a specific role subset (e.g. `facility_manager`, `lecturer`)
- `-num-queries` to control output volume
- `-mode` to support append or overwrite when multiple students contributed to the same file
- `-business-question` to guide the agent to follow each student's intended scenario

The skill was updated with a query distribution table mapping each role to expected query categories: lecturers focused on availability and schedule views; facility staff on approvals, check-in, and maintenance; facility managers on utilization analytics and no-show analysis; department admins on audit trails and inter-department comparisons. This prevented semantic duplication across students and produced a query set with wider operational coverage.

Interaction rule. To protect each student's contribution, the command required the agent to stop and ask the user before overwriting an existing output file. If `-mode` was not specified and the file already existed, the agent would prompt: *"Output file exists. Append new queries for [student name] to the existing file, or overwrite the entire file?"* Only after an explicit answer would the agent proceed. This prevented silent data loss when multiple students ran the command sequentially.

4.7.4 Per-Student Query Summary

Nguyễn Hữu Phước — Role: Facility Staff.

- **Q1 — Pending approvals for today:** Retrieves all pending booking requests within the next 24 hours so facility staff can review and approve or reject them in a timely manner.
- **Q2 — Real-time occupancy snapshot:** Shows which spaces are currently checked-in or approved, who booked them, and when the session ends, enabling live occupancy monitoring across campus.
- **Q3 — Active maintenance tickets:** Lists all open and in-progress maintenance tickets with problem descriptions, assigned staff, and days open, helping staff prioritise repairs.
- **Q4 — Eligible check-in bookings:** Identifies approved bookings past their start time with no check-in, enabling proactive follow-up on no-shows or late arrivals.
- **Q5 — Session completion report for today:** Retrieves sessions completed on a given date with actual end times, space conditions, and check-in staff, supporting end-of-day space review.

Phạm Hữu Nam — Role: Facility Manager.

- **Q6 — Rejected booking audit trail:** retrieves the full rejection history for a specific requester filtered by surrogate key, including rejection reasons, decision timestamps, and the processing staff member. Supports transparency and accountability reviews.
- **Q7 — Maintenance risk profile with upcoming bookings:** identifies spaces with resolved maintenance in the past six months and cross-references each with approved bookings in the next seven days, showing the most recent issue alongside upcoming sessions to support proactive preventive maintenance scheduling.
- **Q8 — Department no-show rate analysis:** aggregates no-show and attended booking counts per department for the semester, computes the no-show rate percentage using a parameterized threshold, and ranks departments from worst to best to support policy enforcement.
- **Q9 — Cumulative usage hours for competing requesters:** identifies requesters with overlapping pending or approved bookings for a contested space and time slot, then calculates their total actual usage hours for the current month using status-aware duration logic, enabling fair-access allocation prioritising low-usage users.
- **Q10 — Room-type utilization summary:** aggregates total requests, approval rates, and unique user counts by space type for a declared semester window, filtering out cancelled bookings, to support data-driven decisions on space portfolio expansion or consolidation.

Cao Quảng Hưng — Role: Student.

- **Q11 — Available spaces with particular facilities:** Finds teaching or study spaces equipped with some particular facilities (such as Projector, WhiteBoard) that are free to book within their desired time window
- **Q12 — Particular events on a date:** Find particular events (such as Seminar or Student Activity) are happening on a given date with specified capacity and organizer.
- **Q13 — Alternative spaces when usual room is blocked:** Learns which room the student uses most from their booking history, checks if it is unavailable then proactively recommends up to a specified number of alternatives on the same floor.
- **Q14 — Lab availability by day of week:** Based on booking history from the past year, retrieves which days of the week have the most available project and computer laboratories
- **Q15 — Department admin's upcoming approved bookings:** A particular depart-

ment administrators identifies all approved upcoming booking made by members of their department, seeing who booked which space, for what purpose, and when.

Trần Đình Quốc Thắng — Role: [Role].

- **Q16 — Lab availability with equipment and conflict checks:** finds computer or project labs for a requested tutorial slot by checking capacity, workstation count, optional projector availability, live room status, confirmed booking overlaps, and unresolved maintenance conflicts.
- **Q17 — Lecturer personal booking timeline:** retrieves a lecturer's semester booking history across all active lifecycle statuses, including room details, decision notes, rejection reasons, approver information, and actual session timestamps.
- **Q18 — Completed TA lab session history:** lists completed computer/project lab sessions for a teaching assistant, including actual duration, condition notes, check-in staff, usage notes, and a summarized facility list.
- **Q19 — Approval lead-time analysis:** aggregates approved and rejected lecturer bookings by purpose, space type, and decision status to show minimum, average, and maximum decision lead time for planning future submissions.
- **Q20 — Upcoming lab readiness alert:** identifies approved upcoming TA lab bookings that may be disrupted by non-available space status or overlapping unresolved maintenance, supporting early backup-room planning.

4.7.5 Issues and Improvements Summary

Table 15 consolidates the significant issues identified across all students during the review cycle, together with the improvements made and their effect on query correctness or maintainability.

Q#	Student	Issue found	Improvement made	Effect
Q6	Nam	Filtered on `full_name`, which is not unique.	Replaced with `DECLARE @requester_id INT`; filter now uses the surrogate key `b.requester_id`.	Eliminates false multi-row matches.
Q9	Nam	`COALESCE(actual_end, requested_end)` overcounted hours for `checked_in` sessions still in progress.	Replaced with a `CASE` expression: `completed` uses `actual_end_time`; `checked_in` in-progress uses `GETDATE()`; month bounds derived via `DATEFROMPARTS`.	Duration accurate per status; query is self-updating each month.

Bảng 15: Issues and improvements across all Task 07 student queries

4.7.6 Evaluation Result

Task 07 was evaluated on query quality, SQL correctness, and alignment with the 4-field template. All queries were compiled against the `CS486_G05` database and verified via `sqlcmd`. Verification output was saved to `logs/eval/task07/`.

Queries that returned zero rows due to sample data coverage gaps were documented with an inline comment noting that the logic was correct and the data seed did not cover the scenario. No query was accepted without at least a syntax verification pass.

5 Conclusion

This project completed a full AI-assisted database design pipeline for the Campus Space Management System. Starting from the original business requirements, the workflow produced a requirement analysis, conceptual ERD, logical schema, validation report, SQL Server implementation, sample data, and business query design. Across these stages, the main database artifacts remained connected through shared registries, design decisions, and traceability records, which helped preserve consistency from the initial problem statement to the final SQL outputs.

The project also demonstrated that an AI agent works more reliably when its context is organized as part of the repository. Global rules in `AGENTS.md`, detailed knowledge in `docs/`, task procedures in `.opencode/commands/` and `.opencode/skills/`, persistent state in `memory/`, and evaluation evidence in `logs/` created a structured workflow rather than a one-time prompting process. This organization reduced context ambiguity and made each task easier to audit and improve.

Through iterative evaluation, the team improved both the generated artifacts and the agent process. Issues such as missing traceability, incomplete validation, inconsistent naming, and insufficient execution evidence were identified and corrected across tasks. Overall, the final result shows that combining database design methods with layered documentation, task-specific skills, and evaluation feedback can produce a more consistent, transparent, and reusable AI-assisted design workflow.

Tài liệu

[1] Our CS486-Project Repository. Link: github.com/AmnO-O/CS486-Project

[2] Trần Văn Dinh. Build app với AI agent: cách mình tổ chức một dự án để AI làm việc đúng convention ngay từ prompt đầu. Viblo link: viblo.asia/p/build-app-voi-ai-agent

[3] DeepEval Documentation. AI Agent Evaluation. Link: deepeval.com/guides/guides-ai-agent-evaluation

[4] Hugo Selbie. A Methodical Approach to Agent Evaluation: Building a Robust Quality Gate. Google Cloud Blog. Link: cloud.google.com/.../a-methodical-approach-to-agent-evaluation

[5] Braintrust. What is agent evaluation? Link: braintrust.dev/articles/agent-evaluation

[6] Openlayer. Agent evaluation complete guide: testing AI agents. Link: openlayer.com/blog/post/agent-evaluation-complete-guide-testing-ai-agents

[7] Daily Dose of Data Science. Six Key Metrics for AI Agent Evaluation. Link: blog.dailydoseofds.com/p/six-key-metrics-for-ai-agent-evaluation

[8] Kore.ai. What is Agent Traceability? - AI Glossary. Link: kore.ai/ai-glossary/what-is-agent-traceability

[9] Claude-Mem Documentation. Progressive Disclosure: Claude-Mem's Context Priming Philosophy. Link: docs.claude-mem.ai/progressive-disclosure

[10] Jekyll Documentation. Front Matter. Link: jekyllrb.com/docs/front-matter

[11] OpenCode Documentation. Commands. Link: opencode.ai/docs/commands

[12] OpenCode Documentation. Agent Skills. Link: opencode.ai/docs/skills

[13] Orlena C. Z. Gotel and Anthony C. W. Finkelstein. An Analysis of the Requirements Traceability Problem. Proceedings of the First IEEE International Conference on Requirements Engineering, 1994. Link: doi.org/10.1109/ICRE.1994.292398